AI LAWYER
FOR EVERYDAY PROBLEMS IN INDIA

Project overview, solution design, technical architecture, quality results, source repository, and hosted application

Weak matches lead to uncertainty; high-risk matters are escalated.

Optional: speech transcription | document drafts | practice conversations

A conversational legal-information prototype that organizes user-provided facts, retrieves relevant project notes, and explains possible next steps. It does not replace a qualified lawyer or provide legal advice.

Project: AI Lawyer India
Source repository: github.com/ayan27mondal/KukuAi_Layer
Hosted application: kukuailayer-d9uc5ewttlsucxor5bcctq.streamlit.app

This report summarizes the project's scope, design, implementation, and observed deployment behavior. It contains no credentials or user case data.

1. Project overview

AI Lawyer India is a Streamlit application designed to make selected legal information easier to navigate for people in India. It collects basic facts about a user's concern, searches a topic-organized knowledge base, and presents possible next steps in plain language. Its workflow combines language-model assistance with retrieval, fixed document templates, and deterministic safety checks.

Scope and user features

Area	Prototype capability	Scope boundary
Security deposit	Fact collection, tenancy-note retrieval, proof checklist, draft notice.	State coverage varies; review drafts.
Consumer issue	Refund, defective product, and service concerns; complaint draft.	Depends on available notes and facts.
Unpaid salary	Basic wage intake, retrieval, evidence checklist, demand notice.	Not a substitute for employment-law advice.
Civil dispute	General information for supported contract, money, and property topics.	No outcome predictions or litigation strategy.
Family matter	Limited general information and escalation guidance.	Information only; consult a qualified lawyer.
Voice and language	Speech input; English, Hindi, Hinglish; Bengali and Assamese depend on model support.	Recognition and language quality can vary.
Practice mode	Role-play and feedback for selected supported scenarios.	Practice feature, not legal representation.

Intended use

The prototype is intended for initial orientation and demonstration. It can help users organize facts and identify questions to raise with a legal professional. It is not designed to decide cases, predict outcomes, or produce filing-ready documents.

2. Solution approach and architecture

The application follows a grounded workflow rather than relying on an unrestricted chat response. Safety screening and retrieval shape the response path, and the system can express uncertainty when the available notes do not support an answer.

Weak matches lead to uncertainty; high-risk matters are escalated.

Optional: speech transcription | document drafts | practice conversations

- The interface accepts typed messages or voice recordings transcribed by a configured provider.
- Deterministic checks identify selected high-risk or out-of-scope situations before a routine answer is prepared.
- The understanding step identifies the likely topic, language, intent, and relevant facts. Missing information can trigger follow-up questions.
- The workflow builds a case summary and searches topic-specific legal notes.
- The answer model receives retrieved notes as context and is instructed to cite them. Weak retrieval leads to an uncertainty response.
- A citation validator checks references against retrieved material before the response is displayed with sources and a disclaimer.
- Where supported, users can generate a template-based draft or enter practice mode.

Technology stack

Layer	Components	Responsibility
Interface	Streamlit; app.py	Conversation, controls, source display, and downloads.
Workflow	pipeline_v2.py; extraction.py; escalation.py	Session flow, fact intake, follow-ups, and escalation.
Knowledge	corpus/; rag.py; rag_chroma.py	Reference notes and keyword or optional vector retrieval.
Models and voice	llm.py; voice.py; Groq; Gemini; gTTS	Understanding, response generation, speech transcription, and audio output.
Validation	validator.py	Checks citations and legal references against retrieved context.
Documents	documents.py; Jinja2; python-docx; ReportLab	Renders template-based DOCX and PDF drafts.
Quality and hosting	pytest; GitHub; Streamlit Community Cloud	Offline regression coverage and hosted application.

Knowledge retrieval

BM25 keyword search is the default retrieval backend and runs over Markdown materials in the project corpus. ChromaDB is an optional vector-search backend that uses Gemini embeddings. The answer flow is designed to avoid filling gaps with unsupported legal claims when relevant notes cannot be found.

3. Safety, privacy, and limitations

Legal information is a high-impact domain. The project includes safeguards to reduce unsupported responses and route selected sensitive situations, but no automated check can guarantee that every answer is complete, current, or legally correct.

Design measure	Role	Limit
Deterministic escalation	Screens selected crisis, criminal, domestic-violence, court-stage, guarantee, and other high-risk messages.	May miss unusual phrasing or flag harmless wording.
Retrieved context	Provides project notes for model-generated explanations.	Coverage and source freshness vary.
Citation validation	Checks cited material and legal references against retrieved notes.	Does not prove correctness or completeness.
Uncertainty handling	Avoids a substantive answer when retrieval is too weak.	Search may miss relevant material.
Document templates	Creates drafts with placeholders for missing information.	Every draft requires careful professional review.
External processing	Sends relevant text, audio, or embeddings to configured providers.	Provider privacy and retention terms apply.

Data and configuration

The application does not use a separate database for chat history; active conversation state is held in the Streamlit session. Corpus content is stored in project files. Generated documents are written to the runtime output directory. Project configuration reads environment variables and loads a local .env file during development. Provider credentials should be stored in an appropriate secret manager and excluded from source control and published materials.

- Responses provide general information, not legal advice or a prediction of outcomes.
- Coverage depends on the state, topic, wording, and available corpus material.
- Laws, procedures, fees, deadlines, and helpline details can change and require authoritative review.
- Generated notices and complaints are drafts, not ready-to-file legal documents.
- Users should avoid sharing sensitive personal information unless they have considered the configured provider's data practices.

4. Repository, deployment, and testing

Source repository

https://github.com/ayan27mondal/KukuAi_Layer/

The public repository contains the application source, documentation, corpus, and tests. Local secrets, generated documents, the local vector index, and test outputs are excluded from the published project. The published commit recorded for the initial deployment is 3726088.

Hosted application

https://kukuailayer-d9uc5ewttlsucxor5bcctq.streamlit.app/

The application is hosted on Streamlit Community Cloud and connected to the public GitHub source, main branch, with app.py as its entry point. The hosted interface was opened and exercised during project verification.

Observed deployment behavior

During a live test on 08 November 2026, the application displayed Groq and Gemini as configured providers and used BM25 retrieval. A Groq request returned HTTP 404 for the model identifier llama-3.3-70b-versatile, with the provider reporting that the model was unavailable or inaccessible. The conversation workflow then used its rule-based fallback, which did not produce a substantive answer to the test question.

The local project configuration was shown as GROQ_MODEL=openai/gpt-oss-120b. The observed cloud response shows that the deployed runtime was not using that model identifier at the time of the test. The application is deployed and accessible, but successful live-model responses were not confirmed.

Verification results

Verification area	Result
Automated offline tests	52 passed in 4.62 seconds in the project virtual environment.
Hosted application access	Streamlit app opened successfully; chat interface rendered.
Provider smoke test	Groq returned HTTP 404 for the deployed model identifier; fallback behavior was observed.
Retrieval mode	BM25 keyword search was active in the hosted application.

5. Project conclusion

AI Lawyer India brings together conversational intake, legal-note retrieval, basic safety routing, source-aware responses, document templates, and practice features in a single hosted prototype. Its design emphasizes clarity about uncertainty and the limits of automated legal information.

The source repository and Streamlit deployment make the project available for review. The offline test suite passed, and the deployed interface is accessible. The live-provider test also identified a model-configuration mismatch: the deployed application attempted to use a Groq model that the provider reported as unavailable or inaccessible. Accordingly, the hosted prototype's interface and fallback flow were demonstrated, but a successful real-model answer was not established in the observed deployment.

Open the project repository

OPEN THE GITHUB REPOSITORY

Direct address: https://github.com/ayan27mondal/KukuAi_Layer/

Scan the QR code to open the same repository.

Hosted application: https://kukuailayer-d9uc5ewttlsucxor5bcctq.streamlit.app/

This project is a software prototype for general legal information. It is not legal advice. People making decisions about a specific matter should consult a qualified legal professional or appropriate legal-aid service.